Lab4 面向对象编程：继承

学习目标:

1. 通过继承现有类来创建类。
2. 基类和派生类的概念以及它们之间的关系。
3. 在继承层次结构中对象构造和析构的顺序。
4. 继承中的初始化。
5. public, protected 和 private 成员访问限定符之间的区别。
6. public, protected 和 private 继承之间的关系。
7. 类成员函数的继承、添加和隐藏。
8. 基类与派生类之间的转换。

实验一：继承体系中对象的构造（Construction）和销毁（Destruction）

1. 创建如下的基类

```
class MyBase1 {
public:
    MyBase1(){ cout << "…BaseClass1 Object is created!"<< end; }
    ~MyBase1(){ cout << "…BaseClass1 Object is destroyed!"<< end; }
}
```

2. 用public继承从MyBase1创建一个派生类并分析输出结果

```
class Myderived1 : public MyBase1 {
public:
    MyDerived1()
    { cout << "…First layer derived Object is created!"<< end; }
    ~MyDerived1()
    { cout << "…First layer derived Object is Destroyed!"<< end; }
}
class Myderived11 : public MyDerived1 {
public:
    MyDerived11()
    { cout << "…Second layer derived Object is created!"<< end; }
    ~MyDerived11()
    { cout << "…Second layer derived Object is destroyed!"<< end; }
```

```
}
int main()
{
    MyBase1 a;
    MyDerived1 b;
    MyDerived11 c;
}
```

3. 创建如下的基类

```
class MyBase2 {
    MyBase1 a1;
public:
    MyBase2()
    { cout << "…BaseClass2 Object is created!"<< end; }
    ~ MyBase2()
    { cout << "…BaseClass2 Object is destroyed!"<< end; }
}
```

4. 用public继承从MyBase2创建一个派生类并分析输出结果

```
class Myderived1 : public MyBase2 {
    MyBase1 a1;
public:
    MyDerived1()
    { cout << "…First layer derived Object is created!"<< end; }
    ~MyDerived1()
    { cout << "…First layer derived Object is Destroyed!"<< end; }
}
class Myderived11 : public MyDerived1 {
public:
    MyDerived11()
    { cout << "…Second layer derived Object is created!"<< end; }
    ~MyDerived11()
    { cout << "…Second layer derived Object is destroyed!"<< end; }
}
int main()
{
    MyBase2 a;
    MyDerived1 b;
    MyDerived11 c;
}
```

实验二：继承中对象的初始化

1. 按如下方式创建2个类并分析输出结果

```
class MyBase31 {
    int a, b, c;
public:
    MyBase31(int x, int y, int z) :a(x), b(y), c(z)
    {
        cout << "…BaseClass31 Object is created!"<< end;
        cout << a << " " << b << " " << c << endl;
    }
    ~ MyBase31(){ cout << "…BaseClass31 Object is destroyed!"<< end; }
}
class MyBase32 {
    int a, b, c;
public:
    MyBase32(int x, int y, int z)
    {
        cout << "…BaseClass32 Object is created!"<< end;
        cout << a << " " << b << " " << c << endl;
        a=x, b=y, c=z;
        cout << a << " " << b << " " << c << endl;
    }
    ~ MyBase32(){ cout << "…BaseClass32 Object is destroyed!"<< end; }
}
int main()
{
    MyBase31 a(1,2,3);
    MyBase32 b(4,5,6);
}
```

2. 按如下方式创建继承类类并分析输出结果

```
class MyDerived1 : public MyBase31 {
    MyBase31 a(5,6,7);
    int c;
public:
    MyDerived1(int x) : c(x), MyBase31(x,8,9)
    {
        cout << "…Base Object has been created!" << endl;
        cout << "…Member Object has been created! " << a.x << " " << a.y << " "
<< a.z << endl;
        cout << "…Derived Object is created! "<< c << endl;
```

```
    }
}
int main()
{
    MyDerived1 b(88);
}
```

实验三：继承中的访问属性

1. 创建一个基类

```
class MyBase3 {
    int x;
    fun1() { cout << “MyBase3---fun1()” << endl; }
protcted:
    int y;
    fun2() { cout << “MyBase3---fun2()” << endl; }
public:
    int z;
    MyBase(int a, int b, int c) {x=a; y=b; z=c;}
    int getX(){cout << “MyBase3---x:” << endl; return x;}
    int getY(){cout << “MyBase3---y:” << endl; return y;}
    int getZ(){cout << “MyBase3---z:” << endl; return z;}
    fun3() { cout << “MyBase3---fun3()” << endl; }
}
```

2. 用public继承从MyBase3创建一个派生类并分析输出结果

```
class MyDerived1 : public MyBase3 {
    int p;
public:
    MyDerived1(int a) : p(a)
        int getP(){cout << “MyDerived---p:” << endl; return p;}
    int disply()
    {
        cout << p << “ “ << x << “ “ << y << “ “ << z << “ “ << endl
            << fun1( ) << endl << fun2() << endl << fun3() << endl;
    }
}
int main()
{
    MyDerived1 a(3);
    a.disply();
```

```
    cout << a.x << “ “ << a.p << “ “ << a.y << “ “ << a.z << endl;
    cout << a.getX() << “ “ << a.getP() << “ “ << a.getY() << “ “ << a.getZ() <<
endl;
}
```

3. 用private继承从MyBase3创建一个派生类并分析输出结果

```
class MyDerived2 : private MyBase3 {
    int p;
public:
    MyDerived2(int a) : p(a)
        int getP(){cout << "MyDerived---p:" << endl; return p;}
    int disply()
    {
        cout << p << " " << x << " " << y << " " << z << " " << endl
            << fun1( ) << endl << fun2() << endl << fun3() << endl;
    }
}
class MyDerived21 : public MyDerived3 {
    int p;
public:
    MyDerived21(int a) : p(a)
        int getP(){cout << "MyDerived21---p:" << endl; return p;}
    int disply1()
    {
        cout << p << " " << x << " " << y << " " << z << " " << endl;
    }

}
int main()
{
    MyDerived2 a(3);
    MyDerived21 b(6);
    a.disply();
    cout << a.x << " " << a.p << " " << a.y << " " << a.z << endl;
    cout << a.getX() << " " << a.getP() << " " << a.getY() << " " << a.getZ() <<
endl;
    b.disply1();
}
```

4. 用protected继承从MyBase3创建一个派生类并分析输出结果

```
class MyDerived3 : protected MyBase3 {
    int p;
```

```
public:
    MyDerived3(int a) : p(a)
        int getP(){cout << "MyDerived---p:" << endl; return p;}
    int disply()
    {
        cout << p << " " << x << " " << y << " " << z << " " << endl
             << fun1( ) << endl << fun2() << endl << fun3() << endl;
    }
}
class MyDerived31 : public MyDerived3 {
    int p;
public:
    MyDerived31(int a) : p(a)
        int getP(){cout << "MyDerived31---p:" << endl; return p;}
    int disply1()
    {
        cout << p << " " << x << " " << y << " " << z << " " << endl;
    }
}
int main()
{
    MyDerived3 a(3);
    MyDerived31 b(6);
    a.disply();
    cout << a.x << " " << a.p << " " << a.y << " " << a.z << endl;
    cout << a.getX() << " " << a.getP() << " " << a.getY() << " " << a.getZ() <<
endl;
    b.disply1();
}
```

5. 分析输出结果

```
class MyBase {
public:
    void f1(){ cout << "…MyBase f1-----!" << endl; }
    void f2(){ cout << "…MyBase f2-----!" << endl; }
}
class MyDerived : public MyBase {
public:
    void f2(){ cout << "…MyDerived f2-----!" << endl; }
    void f22(){ MyBase::f2(); cout << "…MyDerived f2-----!" << endl; }
    void f3(){ cout << "…MyDerived f3-----!" << endl; }
}
```

```
int main()
{
    MyDerived a;
    a.f1(); a.f2(); a.f3(); a.f22();
}
```

实验四：基类与派生类之间的转换

1. 创建一个基类

```
class MyBase {
int x;
public:
MyBase(int a):x(a);
int getX(){ cout << “” << endl; return x; }
}
```

2. 创建一个派生类

```
class MyDerived : public MyBase {
int y;
public:
MyDerived(int a):y(a),MyBase(a+4);
int getY(){ cout << “” << endl; return Y; }
}
```

3. 创建一个测试程序并分析输出结果

```
int main()
{
MyBase a(2), *p = a,;
MyDerived b(4), *q=b;
MyBase &c = a;
MyBase &d = b;
cout << a.getX() << “ ” << p-> getX() << endl;
cout << b.getY() << “ ” << q-> getY() << b.getX() << “ ” << q-> getX() << endl;
a = b;
cout << a.getX() << “ “ << a.getY() << endl;
p = q;
cout << p->getX() << “ “ << p->getY() << endl;
cout << c.getX() << “ “ << d.getX() << “ “ << d.getY() << endl;
b = a;
cout << b.getX() << “ ” << b.getY() << endl;
}
```

实验五：构造和组合

实现 Date 类和 FinalTest 类，使得 main 函数能够正确输出. 所有的成员变量都必须是 private.
提示:
(1) 数据有效性验证不做硬性要求.
(2) 只需实现必要的成员函数.
(3) 接口和实现不必分离.

int main()
{
FinalTest item1("C++ Test", Date(2024,6,2));
item1.print();
FinalTest item2("Java");
item2.print();
item2.setDue(Date(2024,6,10));
item2.print();
}
	Title: C++ Test
	Test Date: 2024-6-2
	Title: Java
	Test Date: 2024-1-1
	Title: Java
	Test Date: 2024-6-10